

VLSI Design Internship

B2B DI 06 - AHB - APB Bridge Project

PROJECT FINAL REPORT

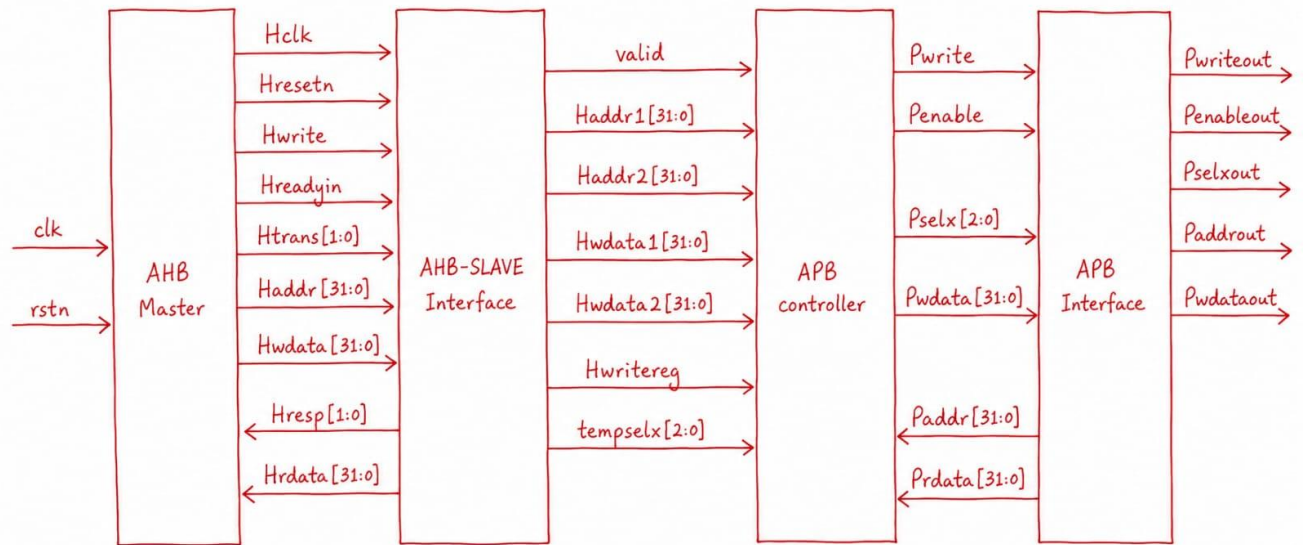
Name: RAKESH V

REG.NO: 24BEC1310

CONTENTS

- 1) Architecture
- 2) Block diagram
- 3) Each blocks with waveforms
- 4) Synthesis
- 5) Conclusion

Architecture



The AHB to APB Bridge acts as an interface between the high-speed Advanced High-performance Bus (AHB) and the low-speed Advanced Peripheral Bus (APB) of the AMBA protocol. The bridge enables communication between an AHB master and multiple APB peripherals by converting AHB transactions into APB-compatible transfers. It ensures that the timing and protocol differences between the two buses are handled efficiently while maintaining reliable data transfer.

The proposed architecture consists of four major functional blocks:

1. AHB Master

The AHB Master initiates all read and write transactions on the AHB bus. It generates the address, write data, control signals, and transfer type required for communication.

2. AHB Slave Interface

The AHB Slave Interface receives transactions from the AHB master and converts them into internal control signals for the APB controller. It captures

the address, write data, transfer type, and write/read information while generating valid requests.

The interface also stores pipeline information such as multiple addresses and write data to support pipelined AHB transfers.

Functions

- Accepts AHB transactions.
- Decodes transfer information.
- Generates the **valid** signal.
- Stores addresses and write data in pipeline registers.
- Generates temporary slave select signals.
- Returns read data, response, and ready signals to the AHB master.

3. APB Controller

The APB Controller is the heart of the bridge. It implements a Finite State Machine (FSM) that converts AHB transactions into APB protocol transactions.

Based on the received control signals, the controller generates the required APB control signals such as **Pselx**, **Penable**, and **Pwrite**. It also controls the timing of setup and enable phases during APB communication.

Functions

- Controls APB transfer sequence.
- Generates APB control signals.
- Supports read and write operations.
- Handles single and burst transfers.
- Generates Hreadyout for synchronization with the AHB bus.

4. APB Interface

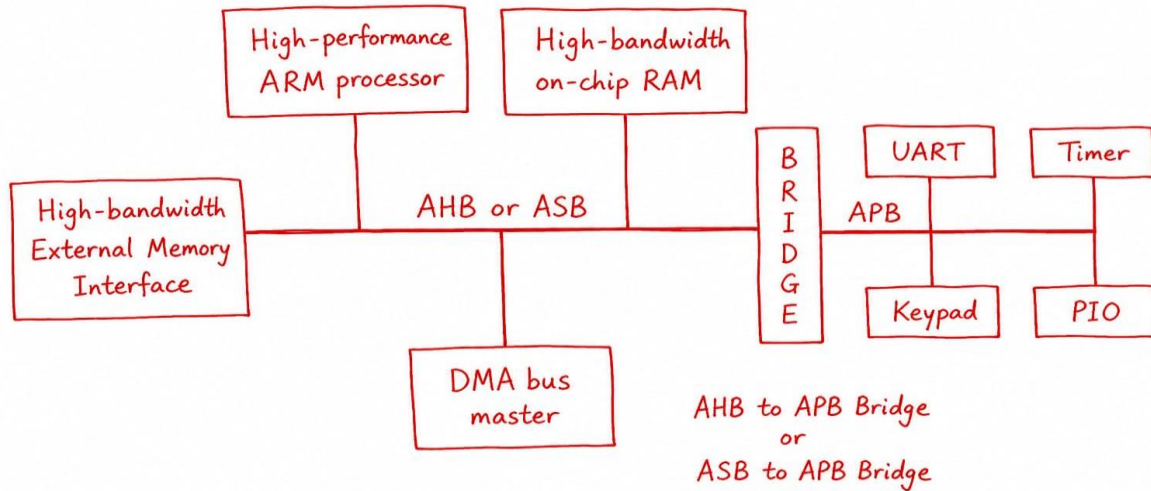
The APB Interface represents the peripheral side of the bridge. It receives the APB control signals generated by the controller and communicates with APB slave devices.

The interface forwards the address, write data, and control signals to the selected peripheral. During read operations, it returns the peripheral data back to the APB controller.

Working of the Architecture

1. The AHB Master initiates a read or write transaction.
2. The AHB Slave Interface captures the address, control, and data signals.
3. A valid request is generated and forwarded to the APB Controller.
4. The APB Controller FSM selects the required APB peripheral and generates **Pselx**, **Penable**, **Pwrite**, **Paddr**, and **Pwdata**.
5. The APB Interface communicates with the selected peripheral.
6. For write operations, data is written directly to the peripheral.
7. For read operations, the peripheral returns **Prdata**, which is transferred back through the APB Controller and AHB Slave Interface as **Hrdata**.
8. The **Hreadyout** signal is asserted after completion of the transaction, indicating to the AHB Master that the transfer has finished successfully.

Block diagram



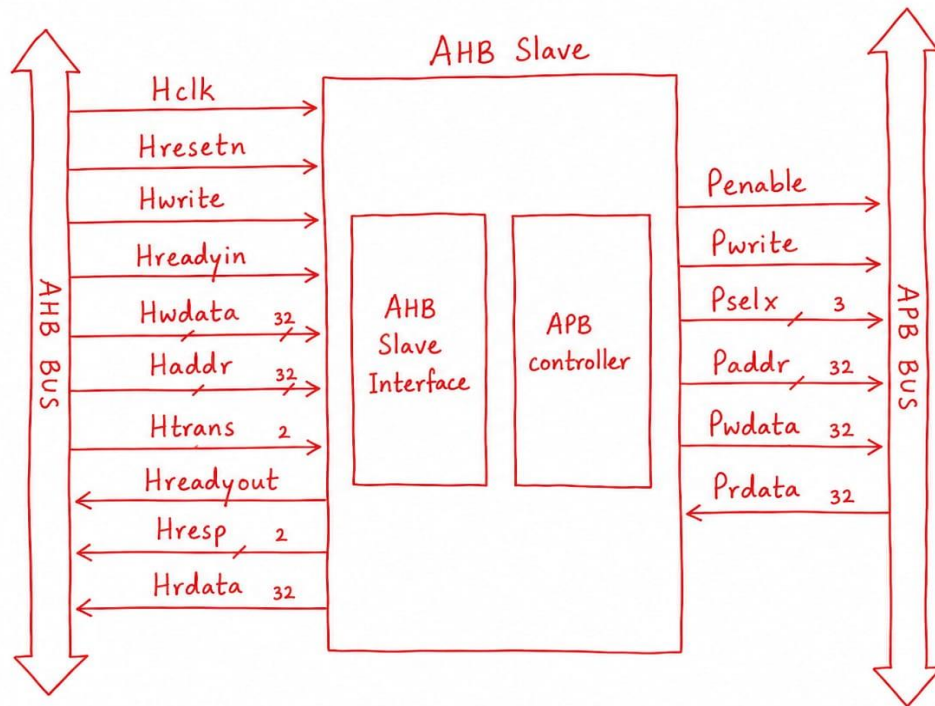
The block diagram represents the functional organization of the AHB to APB Bridge. It illustrates the communication between the AHB Master, AHB Slave Interface, APB Controller, and APB Interface. Each block performs a dedicated function to ensure seamless data transfer between the high-speed AHB bus and the low-speed APB bus.

The AHB Master initiates read and write transactions by generating the address, control signals, and data. These signals are received by the AHB Slave Interface, which captures and decodes the incoming transaction. The decoded information is then forwarded to the APB Controller.

The APB Controller, implemented using a Finite State Machine (FSM), generates the APB control signals such as **PSEL**, **PENABLE**, **PWRITE**, **PADDR**, and **PWDATA**. These signals are transmitted to the APB Interface, which communicates with the selected APB peripheral. During read operations, the APB Interface returns the read data (**PRDATA**) to the controller, which forwards it to the AHB Master through the AHB Slave Interface.

The block diagram simplifies the understanding of signal flow and highlights the interaction among all modules in the bridge.

Block Diagram of AHB to APB Bridge



The Top Module integrates the AHB Slave Interface and APB Controller into a single functional unit, forming the complete AHB to APB Bridge. It provides the external interface for both AHB and APB buses and connects the internal signals between the two modules.

The top module receives all AHB inputs from the AHB Master and forwards them to the AHB Slave Interface. The outputs of the slave interface, including the captured addresses, write data, and control signals, are connected to the APB Controller. The controller generates APB control signals that are sent to the APB Interface and peripherals.

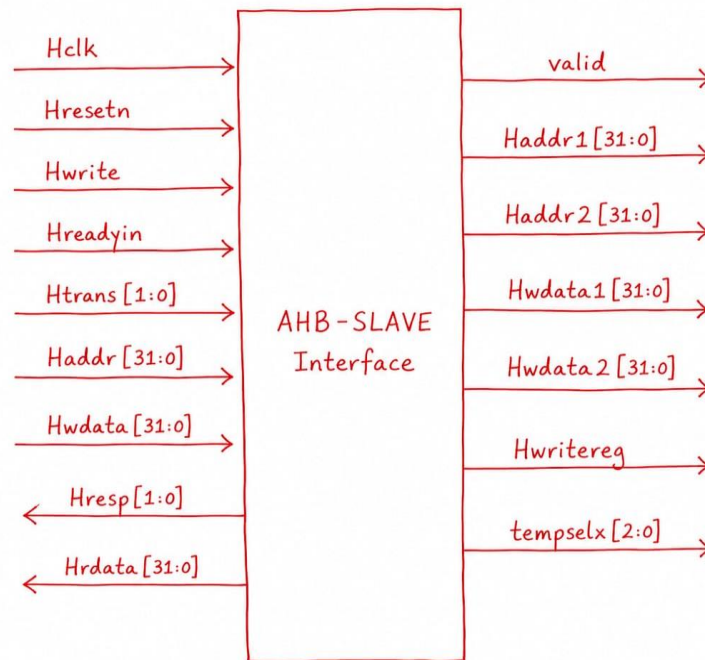
During read operations, data received from the APB peripheral is routed back through the APB Controller and AHB Slave Interface to the AHB Master. The top module also ensures proper synchronization using the common clock and reset signals.

Responsibilities

- Instantiates the AHB Slave Interface.
- Instantiates the APB Controller.
- Connects internal signals between both modules.
- Provides the complete bridge interface.

- Coordinates data flow between AHB and APB buses.
- Serves as the top-level RTL design for synthesis and simulation.

AHB Slave interface



The AHB Slave Interface is responsible for receiving transactions from the AHB Master and converting them into signals required by the APB Controller. It acts as the communication interface between the high-speed AHB bus and the bridge logic.

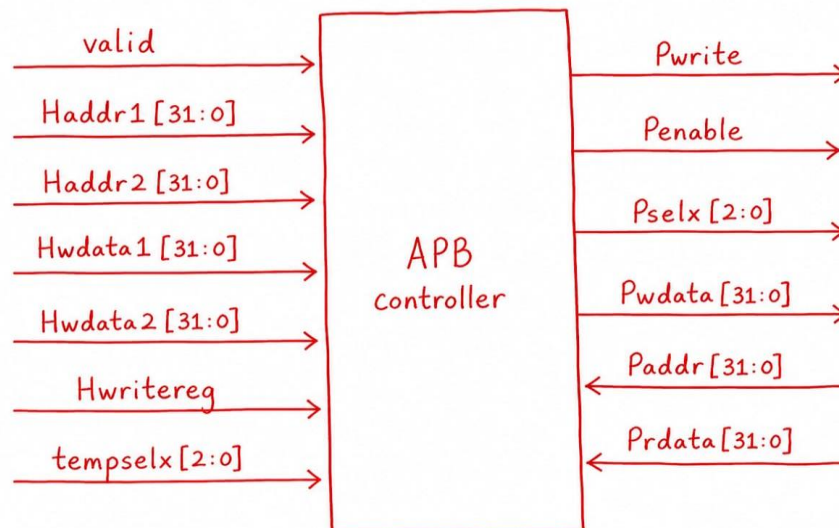
The interface continuously monitors the AHB control signals and validates every incoming transfer. It captures the address, write data, transfer type, and write enable signal. To support pipelined AHB transactions, the interface stores multiple addresses and write data in internal registers before forwarding them to the APB Controller.

The module also generates the **valid** signal, indicating whether the received transaction should be processed. After completion of an APB transaction, it returns the response signals (**HRDATA**, **HRESP**, and **HREADYOUT**) to the AHB Master.

Functions

- Receives AHB read and write requests.
- Decodes transfer type and control signals.
- Stores addresses and write data.
- Generates valid transaction signals.
- Interfaces with the APB Controller.
- Returns read data and response signals to the AHB Master.

APB controller



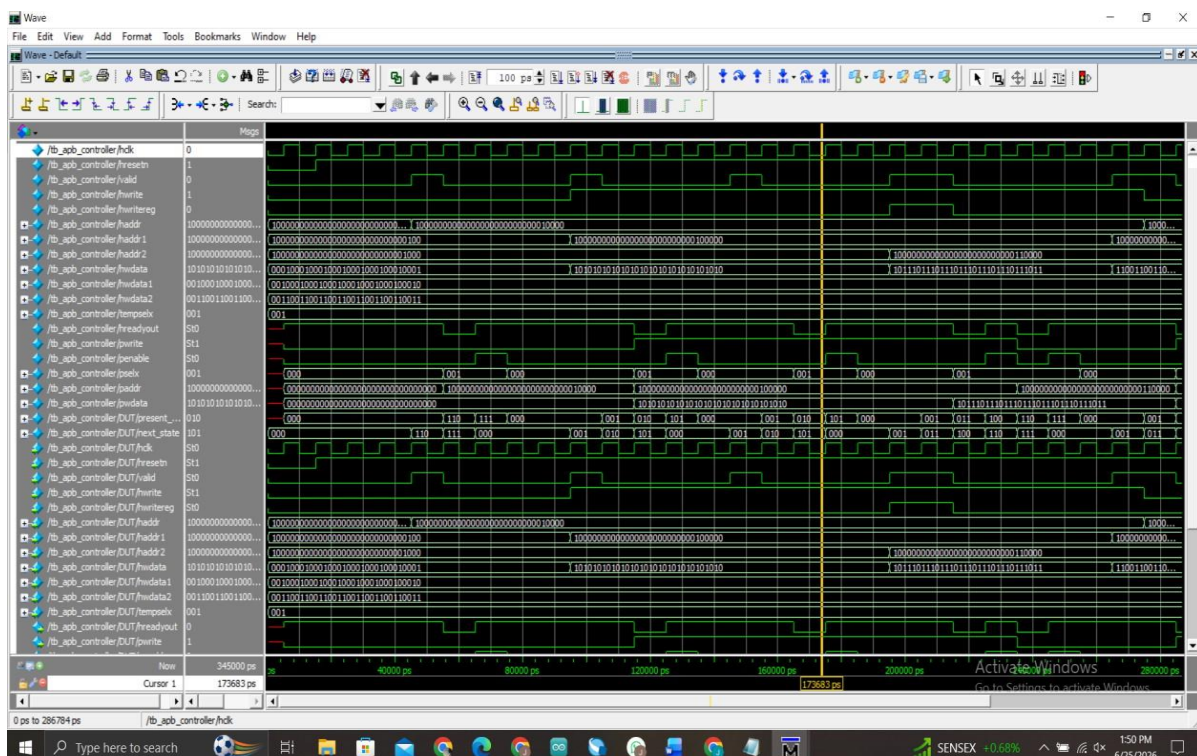
The APB Controller is the central control unit of the bridge. It is implemented as a Finite State Machine (FSM) that converts AHB transactions into APB-compatible transactions.

Depending on the received control signals, the controller moves through different states such as **Idle**, **Setup**, **Enable**, and **Transfer Completion**. During these states, it generates the required APB control signals and ensures that the APB protocol timing is maintained.

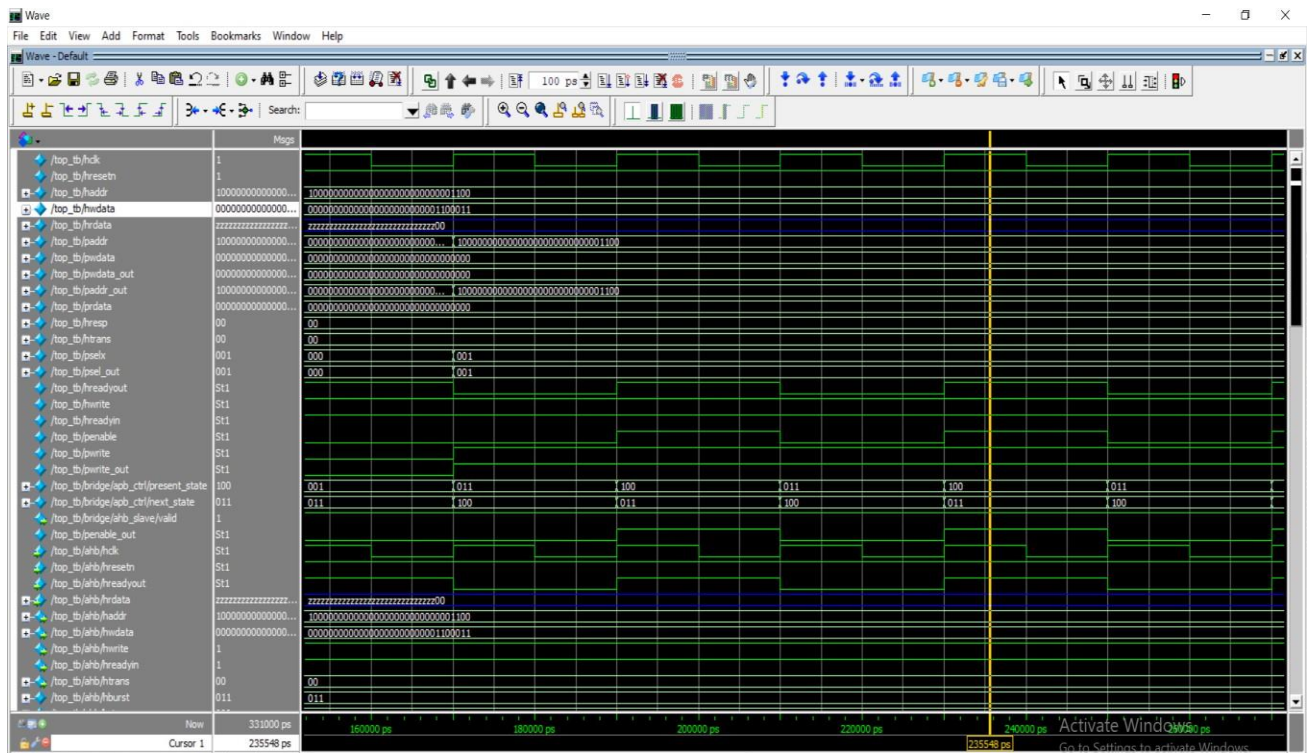
The controller also supports both read and write operations. During write operations, it transfers write data to the selected peripheral. During read operations, it receives data from the APB peripheral and sends it back to the AHB Slave Interface.

Functions

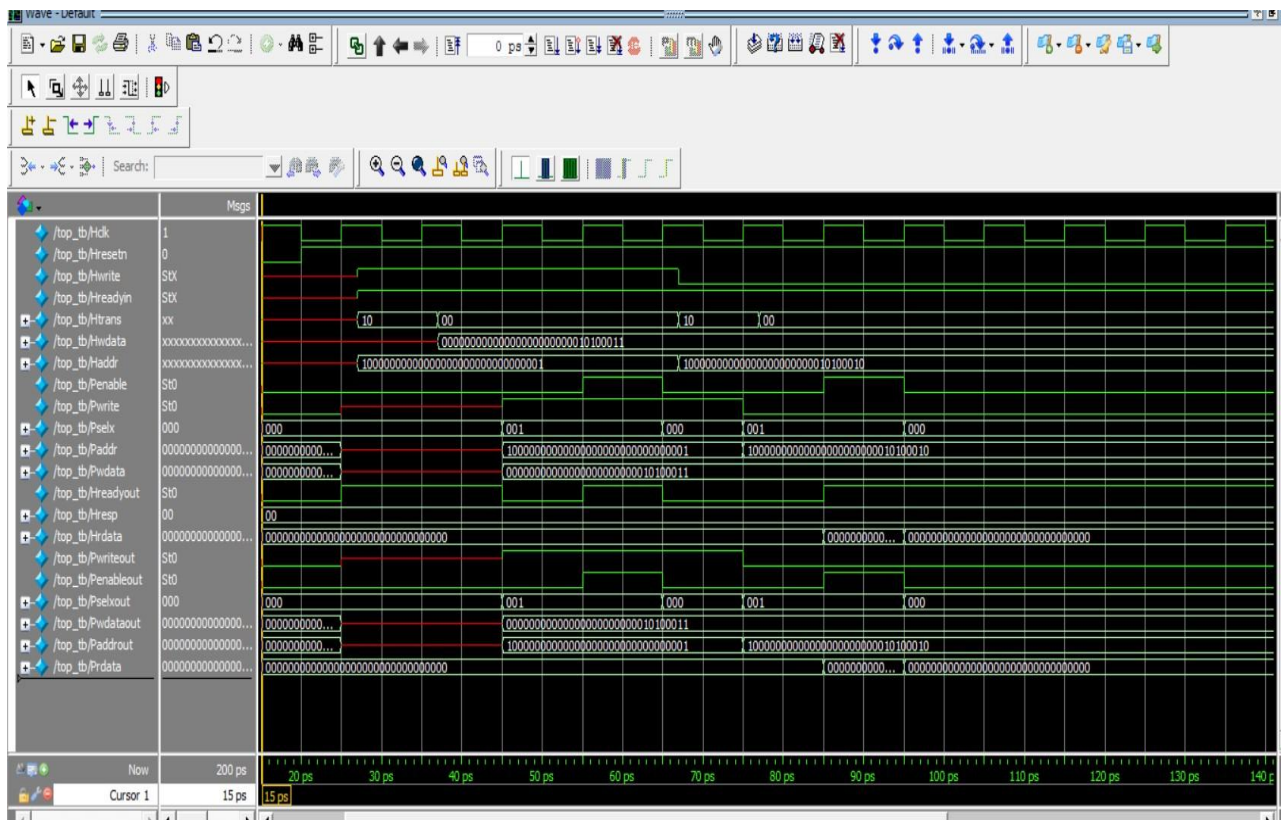
- Implements APB transfer protocol.
- Generates APB control signals.
- Controls read and write operations.
- Synchronizes AHB and APB transactions.
- Generates HREADYOUT signal.
- Supports single and burst transfers.



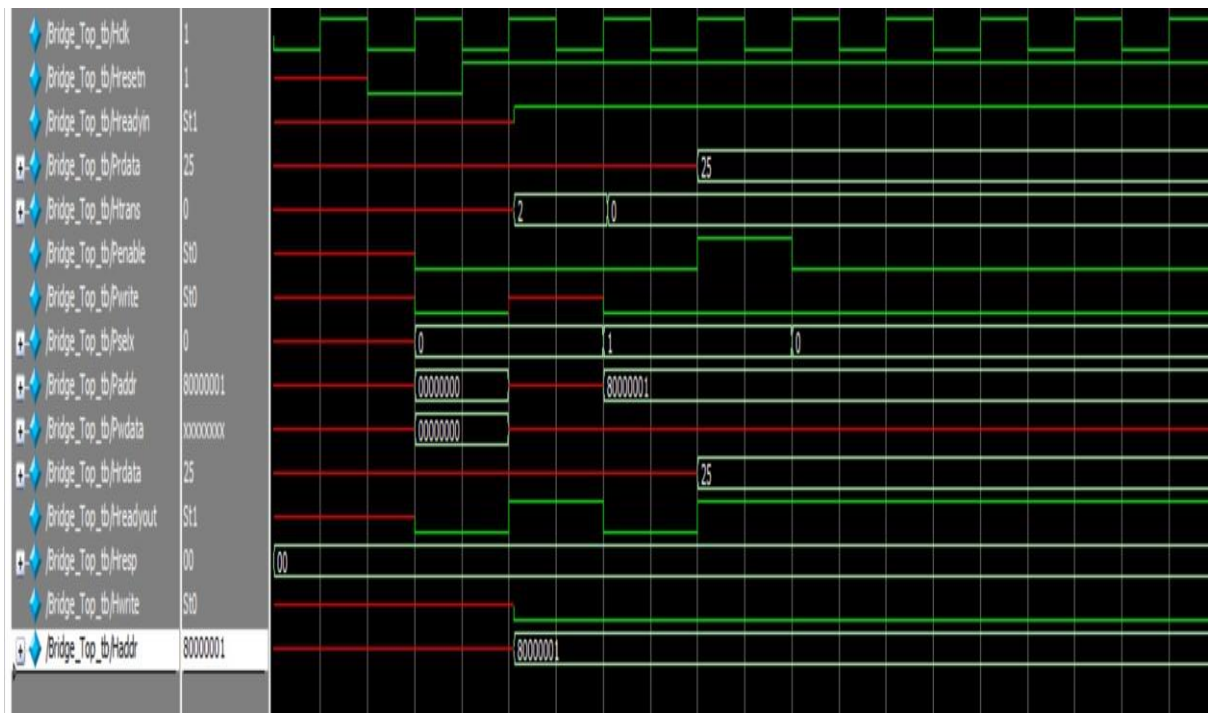
Burst Write :



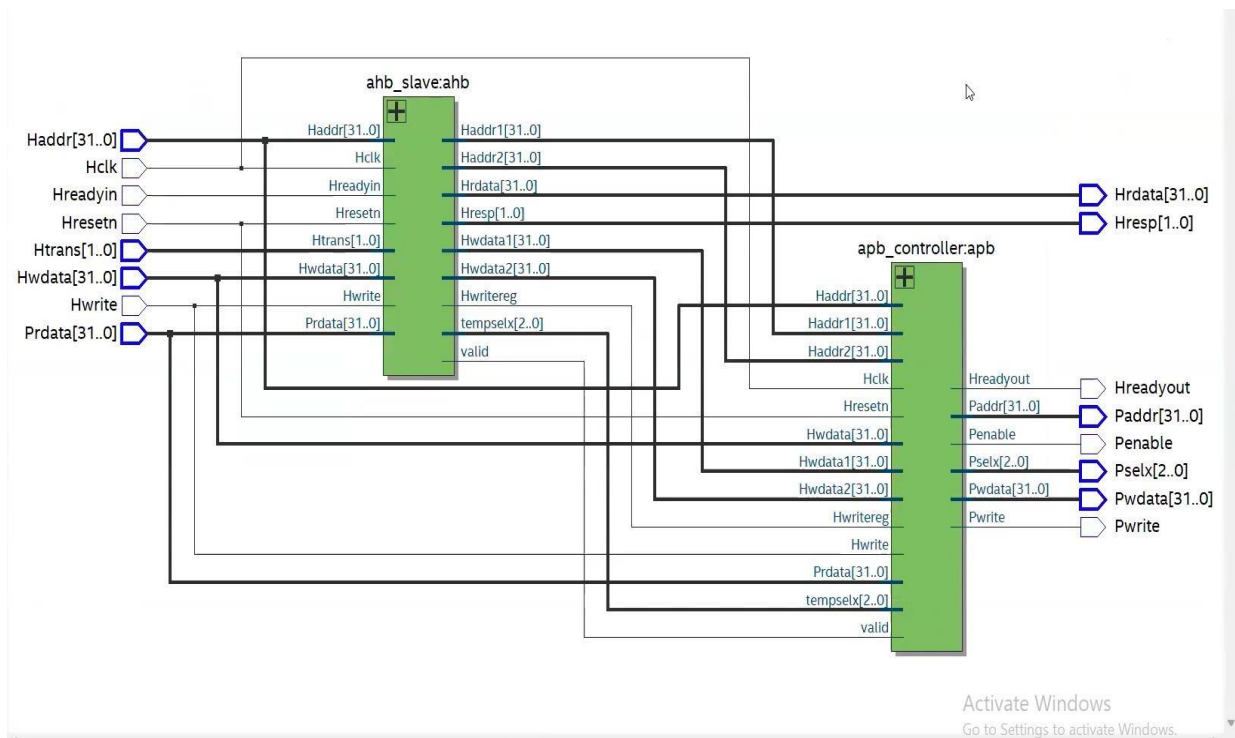
Single Write :



Single Read :



Synthesis



Synthesis is the process of converting the Verilog hardware description into a hardware implementation that can be realized on an FPGA. In this project, the AHB to APB Bridge was synthesized using **Intel Quartus Prime**. During synthesis, the software analyzes the design, verifies the module connectivity, performs logic optimization, and generates the hardware netlist.

The synthesis process confirms that the bridge design is free from synthesis errors and that all modules are correctly interconnected. Quartus Prime also optimizes the logic to achieve efficient hardware utilization while maintaining the intended functionality of the bridge.

The synthesized design consists of two major modules:

- **AHB Slave Interface**
- **APB Controller**

These modules are connected through internal address, data, and control signals to implement communication between the AHB and APB buses.

AHB Slave Interface Synthesis

The AHB Slave Interface module was successfully synthesized using Intel Quartus Prime. The synthesis verifies that the module correctly accepts AHB bus signals such as **HCLK**, **HRESETn**, **HADDR**, **HTRANS**, **HWRITE**, **HWDATA**, and **HREADYIN**.

Based on these inputs, the module generates the internal signals required by the APB Controller

The module also generates the output signals **HRDATA**, **HRESP**, and **HREADYOUT**, which are returned to the AHB Master after transaction completion. The synthesis results confirm that the AHB Slave Interface is correctly implemented and integrated with the remaining bridge logic.

APB Controller Synthesis

The APB Controller was synthesized successfully using Quartus Prime. It receives the address, data, and control information generated by the AHB Slave Interface and controls the APB transfer sequence through its Finite State Machine (FSM).

During synthesis, the controller is mapped into hardware logic capable of generating the APB control signals:

- PWRITE
- PENABLE
- PSELX[2:0]
- PADDR[31:0]
- PWDATA[31:0]

The controller also receives **PRDATA** from the APB peripheral during read operations and generates **HREADYOUT** to synchronize the completion of each transaction with the AHB Master. The synthesis confirms that all control paths and data paths are correctly implemented.

Synthesis Analysis

The synthesis diagram generated by Intel Quartus Prime illustrates the hardware implementation of the AHB to APB Bridge. It clearly shows the interconnection between the **AHB Slave Interface** and the **APB Controller** through internal address, data, and control signals.

From the synthesis results, it can be observed that:

- The AHB Slave Interface receives all AHB bus inputs.
- Internal address, data, and control signals are correctly connected to the APB Controller.
- The APB Controller generates all required APB control signals.
- Read data, response, and ready signals are correctly returned to the AHB Master.
- All modules are successfully synthesized and interconnected without synthesis errors.

The successful synthesis verifies that the proposed AHB to APB Bridge design is suitable for hardware implementation and further functional verification.

Conclusion

The AHB to APB Bridge was successfully designed and implemented using Verilog HDL to enable communication between the high-speed AHB bus and the low-speed APB bus. The design included the AHB Slave Interface, APB Controller, and Top Module, which together ensured accurate protocol conversion and reliable data transfer between the two bus architectures.

The functionality of the design was verified through simulation by testing different operations such as single read, single write, burst read, and burst write transactions. The design was then synthesized using Intel Quartus Prime, confirming that it is free from synthesis errors and suitable for hardware implementation.

This project provided a practical understanding of AMBA bus protocols, finite state machine (FSM) design, RTL coding, digital system design, simulation, and synthesis. It also enhanced my problem-solving skills and strengthened my knowledge of VLSI design methodologies. Overall, the successful implementation of the AHB to APB Bridge serves as a strong foundation for developing more complex SoC communication systems and future digital hardware desi